

实验 3 - 卷积神经网络

数据科学与大数据技术专业

1 实验目标

- 理解卷积神经网络 (CNN) 中局部连接、权值共享、感受野、填充、步幅和池化的作用.
- 使用 NumPy 从零实现卷积层和池化层的前向传播, 并理解输出尺寸的计算方式.
- 选做实现卷积层和池化层的反向传播, 理解梯度如何传回输入、滤波器和偏置.
- 使用 PyTorch 构建卷积神经网络, 完成 Happy House 二分类和 SIGNS 手语数字多分类任务.
- 掌握 `nn.Sequential` 与自定义 `nn.Module` 两种模型组织方式的差异.
- 熟悉图像张量在 NumPy 与 PyTorch 中的形状转换, 尤其是 NHWC 与 NCHW 格式.

2 实验环境

- 操作系统: Windows / Linux / macOS
- Python 3.7 及以上
- 主要依赖库: `numpy`、`matplotlib`、`h5py`、`torch`
- 推荐开发环境: Jupyter Notebook 或 VS Code
- 实验文件:
 - W6A1/Convolution_model_Step_by_Step_v1.ipynb
 - W6A2/Convolution_model_Application_PyTorch.ipynb

如本地环境缺少依赖, 可在终端中执行:

```
pip install numpy matplotlib h5py torch
```

3 背景知识

3.1 卷积神经网络的基本思想

全连接网络在处理图像时会把图像展平为长向量, 这会破坏像素的空间邻接结构, 并导致参数量迅速增大. 卷积神经网络通过局部连接和权值共享来直接处理图像张量, 使模型能够学习边缘、纹理、局部形状等空间模式.

对输入图像体积 $A_{\text{prev}} \in \mathbb{R}^{n_H \times n_W \times n_C}$, 一个卷积滤波器 $W_c \in \mathbb{R}^{f \times f \times n_C}$ 在某个空间位置上计算:

$$z = \sum_{i=1}^f \sum_{j=1}^f \sum_{k=1}^{n_C} A_{\text{slice}, i, j, k} W_{c, i, j, k} + b_c \quad (3.1)$$

同一个滤波器会滑过整张图像, 产生一个二维特征图. 多个滤波器堆叠后形成多通道输出.

3.2 填充、步幅与输出尺寸

填充 (Padding) 是在图像边界补零. 它可以保留边缘信息, 也可以控制卷积后的空间尺寸. **步幅 (Stride)** 表示滑动窗口每次移动的像素数.

若输入高度和宽度分别为 $n_{H_{\text{prev}}}$ 、 $n_{W_{\text{prev}}}$, 滤波器大小为 f , 填充为 p , 步幅为 s , 则卷积输出尺寸为:

$$n_H = \left\lfloor \frac{n_{H_{\text{prev}}} - f + 2p}{s} \right\rfloor + 1 \quad (3.2)$$

$$n_W = \left\lfloor \frac{n_{W_{\text{prev}}} - f + 2p}{s} \right\rfloor + 1 \quad (3.3)$$

$$n_C = \text{滤波器个数} \quad (3.4)$$

本实验第一部分的数据格式采用 NumPy 常见的 NHWC: (m, n_H, n_W, n_C) .

3.3 池化层

池化层没有可学习参数, 主要用于降低空间尺寸和增强局部平移不变性. 最大池化取窗口内最大值:

$$A_{h, w, c} = \max A_{\text{prev}}[h : h + f, w : w + f, c] \quad (3.5)$$

平均池化取窗口均值:

$$A_{h, w, c} = \frac{1}{f^2} \sum A_{\text{prev}}[h : h + f, w : w + f, c] \quad (3.6)$$

若池化窗口大小为 f , 步幅为 s , 且不使用填充, 输出尺寸为:

$$n_H = \left\lfloor \frac{n_{H_{\text{prev}}} - f}{s} \right\rfloor + 1, \quad n_W = \left\lfloor \frac{n_{W_{\text{prev}}} - f}{s} \right\rfloor + 1, \quad n_C = n_{C_{\text{prev}}} \quad (3.7)$$

3.4 卷积与池化的反向传播

卷积层反向传播需要把输出梯度 dZ 分别累加到输入、权重和偏置上. 对某一输出位置和滤波器 c :

$$dA_{\text{prev}, \text{slice}} += W_c dZ_{h, w, c} \quad (3.8)$$

$$dW_c += A_{\text{prev}, \text{slice}} dZ_{h, w, c} \quad (3.9)$$

$$db_c += dZ_{h, w, c} \quad (3.10)$$

最大池化反向传播只把梯度传给窗口中最大值所在的位置; 平均池化反向传播则把梯度均匀分配给窗口内所有元素.

3.5 PyTorch 中的图像张量格式

PyTorch 的卷积层 `nn.Conv2d` 接收 NCHW 格式, 即 (N, C, H, W) . 而本实验数据集通常由 H5 文件读入为 NHWC, 即 (N, H, W, C) . 因此输入 PyTorch 前需要转置:

```
X_t = torch.tensor(X_np.transpose(0, 3, 1, 2), dtype=torch.float32)
```

- 二分类任务可使用 Sigmoid 输出并配合 BCELoss.
- 多分类任务通常直接输出 logits, 并使用交叉熵损失; 该损失内部已经包含 $\log \text{softmax}$.

4 实验步骤

实验 3.1: NumPy 实现卷积网络基础模块 (W6A1)

实验目标: 不依赖深度学习框架, 用 NumPy 实现 CNN 的基础运算, 明确每个循环变量和张量形状的含义.

3.1.1 零填充

练习 1: zero_pad

对一批图像 $X \in \mathbb{R}^{m \times n_H \times n_W \times n_C}$ 在高度和宽度维度补零:

$$X_{\text{pad}} \in \mathbb{R}^{m \times (n_H+2p) \times (n_W+2p) \times n_C} \quad (4.1)$$

实现要点:

- 使用 `np.pad`.
- 只填充第 2、3 个维度, 不填充样本维和通道维.
- 推荐写法: `((0,0),(pad,pad),(pad,pad),(0,0))`.

3.1.2 单步卷积

练习 2: conv_single_step

实现单个窗口和单个滤波器的卷积:

$$Z = \sum (a_{\text{slice}} \odot W) + b \quad (4.2)$$

其中 a_{slice} 与 W 形状均为 (f, f, n_C) . 实现时先逐元素乘法, 再 `np.sum`, 最后加上偏置并转为标量.

3.1.3 卷积前向传播

练习 3: conv_forward

将单步卷积推广到整个 batch、所有空间位置和所有滤波器. 核心循环为:

- 遍历样本 i .
- 遍历输出高度 h 与宽度 w .
- 根据 `stride` 计算 `vert_start`、`vert_end`、`horiz_start`、`horiz_end`.
- 遍历滤波器通道 c , 调用 `conv_single_step`.

函数返回卷积输出 Z 和反向传播需要的 `cache=(A_prev,W,b,hparameters)`.

3.1.4 池化前向传播

练习 4: `pool_forward`

实现池化层的前向传播, 支持:

- `mode="max"`: 最大池化.
- `mode="average"`: 平均池化.

池化层对每个通道独立操作, 不改变通道数. 实现时要注意池化窗口只覆盖高度和宽度, 不跨通道.

3.1.5 反向传播 (选做)

选做练习包括:

- `conv_backward`: 计算 `dA_prev`、`dW`、`db`.
- `create_mask_from_window`: 为最大池化反向传播生成最大值掩码.
- `distribute_value`: 为平均池化反向传播均匀分配梯度.
- `pool_backward`: 整合最大池化与平均池化的反向传播.

完成选做时要特别处理 `pad=0` 的情况, 避免用空切片覆盖 `dA_prev`.

实验 3.2: PyTorch 卷积网络应用 (W6A2)

实验目标: 使用 PyTorch 搭建、训练和评估卷积神经网络, 比较 `nn.Sequential` 与自定义 `nn.Module`.

3.2.1 Happy House 二分类

Happy House 使用 64×64 的 RGB 图像, 标签表示图片中的人是否微笑. 模型使用顺序容器构建:

$$\text{ZeroPad2d} \rightarrow \text{Conv2d} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d} \rightarrow \text{Flatten} \rightarrow \text{Linear} \rightarrow \text{Sigmoid} \quad (4.3)$$

关键形状推导:

- 输入: $(N, 3, 64, 64)$.
- `ZeroPad2d(3)` 后: $(N, 3, 70, 70)$.

- `Conv2d(3,32,kernel_size=7,stroke=1)` 后: $(N, 32, 64, 64)$.
- `MaxPool2d(2,2)` 后: $(N, 32, 32, 32)$.
- 展平维度: $32 \times 32 \times 32 = 32768$.

训练时使用 `nn.BCELoss()` 和 Adam 优化器. 预测时以 0.5 为阈值计算二分类准确率.

3.2.2 SIGNS 手语数字多分类

第二个任务是识别 0 到 5 的手语数字. 使用自定义 PyTorch 模块, 主体结构为两个“卷积 + ReLU + 池化”模块, 接展平层和线性分类层. 推荐结构:

- `Conv2d(3,8,kernel_size=4,stroke=1,padding='same')`.
- `MaxPool2d(kernel_size=8,stroke=8)`, 空间尺寸 $64 \rightarrow 8$.
- `Conv2d(8,16,kernel_size=2,stroke=1,padding='same')`.
- `MaxPool2d(kernel_size=4,stroke=4)`, 空间尺寸 $8 \rightarrow 2$.
- Flatten 后维度为 $16 \times 2 \times 2 = 64$.
- `Linear(64,6)` 输出 6 类 logits.

多分类训练使用 `nn.CrossEntropyLoss()`. 标签应为整数类别索引, 不要转为 one-hot; 模型最后也不要加 softmax.

3.2.3 训练循环与结果分析

PyTorch 标准训练循环包括:

1. `optimizer.zero_grad()` 清空梯度.
2. 前向传播得到输出或 logits.
3. 计算损失.
4. `loss.backward()` 自动反向传播.
5. `optimizer.step()` 更新参数.

请记录训练集和测试集的损失、准确率, 并绘制训练历史曲线. 若训练准确率显著高于测试准确率, 应分析是否过拟合.

5 核心函数与模块总结

函数 / 模型	所在实验	作用
zero_pad	W6A1	对图像 batch 做零填充
conv_single_step	W6A1	单窗口单滤波器卷积
conv_forward	W6A1	卷积层前向传播
pool_forward	W6A1	最大 / 平均池化前向传播
conv_backward	W6A1 选做	卷积层反向传播
pool_backward	W6A1 选做	池化层反向传播
happyModel	W6A2	nn.Sequential 二分类 CNN
ConvolutionalModel	W6A2	自定义 nn.Module 多分类 CNN

6 实验作业

1. 完成 W6A1 中 zero_pad、conv_single_step、conv_forward、pool_forward 的代码, 并通过测试单元格.
2. 选做: 完成 conv_backward、create_mask_from_window、distribute_value、pool_backward.
3. 完成 W6A2 中 HappyModel 与 SIGNS 多分类模型的搭建、训练和评估.
4. 在实验记录中说明 NHWC 到 NCHW 的转换原因, 并给出两个任务的损失曲线和准确率.
5. 比较 nn.Sequential 与自定义 nn.Module 的适用场景.
6. 选做: 寻找一个小型 CNN 开源项目或公开教程, 在本地复现并用文字说明模型结构、数据集和运行结果.

7 提交要求

请在提交前检查材料是否齐全. 本课程作业上传只接受文本文件, 不接受 PDF、Word 文档、图片、截图、压缩包等二进制文件. 采用这一规则, 一方面是为了引入 AI 辅助评阅, 另一方面也是希望大家养成用清晰文本表达实验过程和结论的习惯.

如果实验报告确实需要配图, 请优先思考是否可以用文字、公式或表格表达清楚; 若必须插图, 请将图片放入 Notebook 中, 形成一份完整的 .ipynb 实验报告. 若使用 L^AT_EX 或 Word 撰写报告, 请使用 pandoc 等工具转换为 .md 或 .txt 等文本格式后再提交, 不要提交 .pdf 或 .docx.

1. Jupyter Notebook 文件 (.ipynb)

- (a) 必交内容包括 W6A1 和 W6A2 对应的 Notebook 文件.
- (b) Notebook 中应包含完整代码、运行结果、损失曲线、准确率输出和必要的分析文字.
- (c) 只需提交 Notebook 源文件, 不需要额外导出为 PDF、HTML、Python 脚本、Word 文档、截图或图片文件.

2. 实验过程与 AI 互动记录文本文件 (.md / .txt)

- (a) 必须额外提交一份文本文件, 作为实验过程与 AI 互动记录, 推荐使用 .md 或 .txt.

- (b) 记录内容至少包括: 查阅了哪些资料、向 AI 提出了哪些问题、如何继续思考、调试、修改代码或完善实验分析.
- (c) 不要把 AI 的回复原文放入记录中, 也不要复制粘贴大段代码; 记录应只包含你自己写的内容, 包括你的问题、判断、尝试、修改思路和必要的简短总结.
- (d) 记录应尽量按时间顺序整理, 不要只保留最终答案.

8 关于作业的重要说明

本实验的核心目标是理解卷积和池化的底层计算, 以及 PyTorch 如何把这些计算封装为可训练模块. 代码填空部分应由学生自己完成, 并通过调试和单元测试逐步确认正确性.

可以使用 AI 工具辅助理解概念、解释报错、查询 API 用法或检查张量形状, 但不得直接让 AI 代写全部必做代码后不加理解地提交. 若使用 AI 工具, 必须在文本记录中如实写明自己的提问、自己的判断和后续修改过程.

AI 互动记录应只保留学生自己写的内容. 不要粘贴 AI 的完整回复, 不要粘贴复制来的大段代码, 也不要把聊天记录原样导出后提交.

选做内容可以更自由地使用 AI 工具, 不需记录使用过程。